

```
int N;
N=10;
#pragma omp parallel {
#pragma omp for
for (int i=0; i<N; i++)
printf("I am from loop one %d\n",i);
#pragma omp barrier
#pragma omp for
for (int i=0; i<N; i++)
printf("I am from loop two %d\n",i+50);
}
```

Since we have used 'barrier', the second loop waits till the first loop execution is complete.

The next feature is the 'master'. We will see how we can assign a task to the master thread. The master thread is basically the thread that is calling the parallel pieces of code. We can assign specific tasks to this thread -- mostly verification tasks. In the following example, we give a generic task to all the threads — the parallel tasks. To the master we will give the task to print the total number of threads that are there. This can be done using the following piece of code and the output can be seen in Figure 4.

```
#include <omp.h>
```

```
#define _GNU_SOURCE
#include<stdio.h>
#include <sched.h>
int main()
{
#pragma omp parallel
{
printf("Hello world!!\n");
#pragma omp master
printf("%d\n", omp_get_num_threads());
}
}
```

Here we made the threads print 'Hello world!!' and the master thread prints the total threads that are there.

The next feature that we are going to look at is 'critical'. This feature is useful when we are working with shared variables. Basically, if we need to update a variable more than once in the code and if we run those lines of code parallelly, then there is a chance that the variable gets updated in the wrong sequence. Therefore, the end output may be wrong. Using this 'critical' feature, we can ask the second updation command to run only after the first one is complete. Let us look at the code to add x and 1 and

update the value of x to x+1. We will look at the outputs using and not using 'critical' to understand the difference.

You can run the following code to see the output without using 'critical':

```
#include <omp.h>
#define _GNU_SOURCE
#include<stdio.h>
#include <sched.h>
int main()
{
int x; x = 0;
#pragma omp parallel shared(x)
{
x = x + 1;
printf("%d\n",x);
}
```

Here, updates are being carried out simultaneously as we are not using 'critical'. Hence, the value of x is the same more than once as more than one thread may be updating it at the same time. Had we used 'critical', the updations would have been executed one after another.

Now let's apply the 'critical' feature, using the following code:

```
#include <omp.h>
#define _GNU_SOURCE
#include<stdio.h>
#include <sched.h>
int main()
{
int x;
x = 0;
#pragma omp parallel shared(x) {
#pragma omp critical
x = x + 1;
printf("%d\n",x); }
```

Here we are using 'critical', so all the updates of x are being done one after the other. As you can see in Figure 6, each of the values of x is unique.

Another advanced feature of OpenMP is 'single'. Using this feature, we can assign tasks to the

```
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$ gcc -o filename -fopenmp abc.c
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$ ./filename
Hello world!!
4
Hello world!!
Hello world!!
Hello world!!
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$
```

Figure 4: 'Master' thread use output

```
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$ gcc -o filename -fopenmp abc.c
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$ ./filename
3
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$
```

Figure 5: Output without 'critical'

```
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$ gcc -o filename -fopenmp lab5.c
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$ ./filename
4
11
12
jishnusaaurav@jishnusaaurav-IdeaPad-L340-15IRH-Gaming:~/Desktop$
```

Figure 6: Output with 'critical'